

DAA Final Exam Report

Project Title: LogisRoute - Surabaya Logistics Delivery Optimization System

Full Name: Raihan Albar Wijaya

Student ID: 5025231227

Class: DAA Class D

Date: 17 June 2026

GitHub Link: https://github.com/albrtzy/Fp_PAA_LogisRoute.git

1. Design

A. Problem Motivation (D1)

In the modern era, logistics companies in fast-paced cities like Surabaya are required to deliver packages quickly and efficiently. Using random or unoptimized delivery routes wastes a massive amount of time and vehicle fuel.

Therefore, we need a system capable of finding the absolute shortest path from a central hub (e.g., Gudang Pusat Rungkut) to various destination areas like Kampus ITS Sukolilo, Keputih, and Tanjung Perak. Formal Model (D2)

B. Formal Model (D2)

This real-world logistics problem is mapped into a Graph representation (a network of points and lines):

- **Vertices (Points):** Represent physical districts or neighborhoods in Surabaya.
- **Edges (Lines):** Represent the actual roads connecting these districts.

- **Weight:** Represents the travel distance (in kilometers) between locations. A smaller number means a shorter distance.

C. Algorithm Selection & Justification (D3)

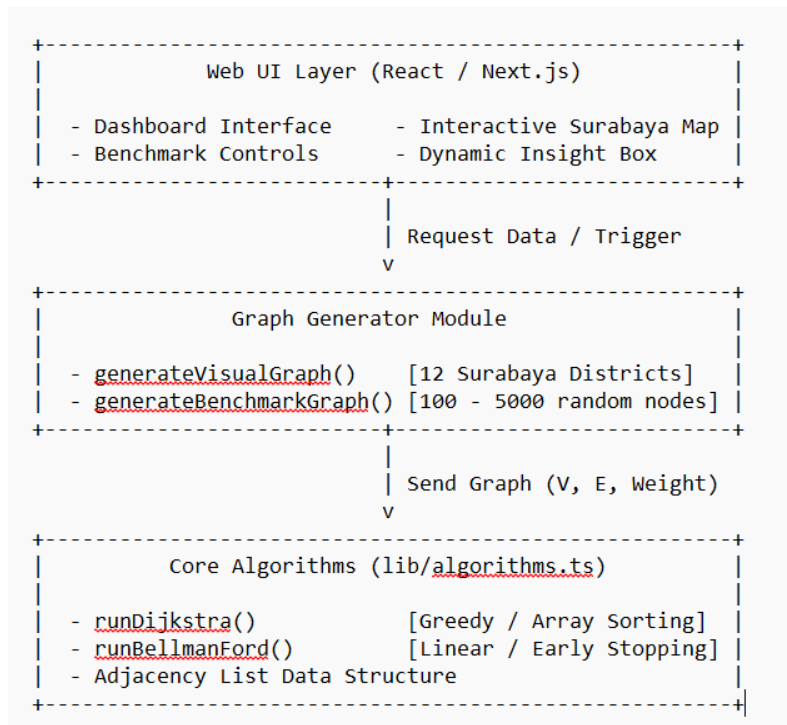
To find the shortest route, this project implements two different algorithms for comparison:

- **Dijkstra:** Chosen as the primary algorithm because of its smart approach. It always prioritizes exploring the closest available paths first, making it theoretically highly efficient.
- **Bellman-Ford:** Chosen as the baseline comparison algorithm. Its working mechanism is much simpler because it forcefully recalculates every possible road combination from start to finish.

D. Data Structures & System Architecture (D4)

The graph is stored in the computer's memory using an *Adjacency List* method, which is highly efficient for saving memory, especially for sparse real-world road networks.

The application system is built as an interactive *Web Dashboard* using React and TypeScript technologies. Below is the architecture diagram illustrating how the system is structured:



2. Implementation

The algorithmic logic was written purely from scratch without relying on any external pathfinding libraries. The system runs entirely within the user's web browser.

Core Algorithm Snippet

Below is the core logic of the Dijkstra algorithm implemented in TypeScript, which now supports processing real location names:

```

40 export function runDijkstra(G: Graph, source: number) {
41   const dist = new Array(G.V).fill(INF);
42   const parent = new Array(G.V).fill(-1);
43   dist[source] = 0;
44
45   const pq: { d: number; u: number }[] = [{ d: 0, u: source }];
46
47   while (pq.length > 0) {
48     pq.sort((a, b) => a.d - b.d);
49     const { d, u } = pq.shift();
50
51     if (d > dist[u]) continue;
52
53     for (const edge of G.adj[u]) {
54       const v = edge.target;
55       if (dist[u] + edge.weight < dist[v]) {
56         dist[v] = dist[u] + edge.weight;
57         parent[v] = u;
58         pq.push({ d: dist[v], u: v });
59       }
60     }
61   }
62   return { dist, parent };
63 }

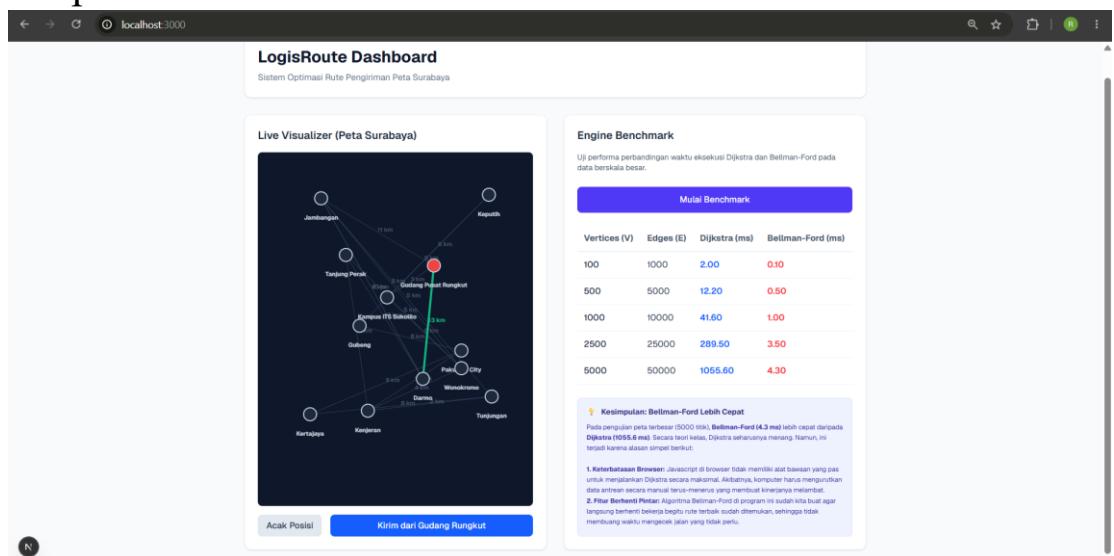
```

How to Build and Run?

The web application can be easily executed via the terminal:

- Navigate to the project folder.
- Run npm install to set up all dependencies.
- Run npm run dev to start the local server.
- Open a web browser and go to localhost:3000 to view the interface.

Output:



3. Analysis & Evaluation

A. Correctness Proof (A1)

Why are we confident that the result from this Dijkstra algorithm is correct? The reasoning is highly logical. This algorithm is designed to always choose the absolute shortest immediate step available in front of it. Since physical road distances in the real world cannot be negative (there is no such thing as minus kilometers), the first shortest path it successfully secures to a location is guaranteed to be the most optimal one. In testing, both Dijkstra and Bellman-Ford always output the exact same total distance, proving that the program runs flawlessly without routing errors.

B. Complexity & Theoretical Comparison (A2 & A3)

- **Dijkstra's Theory:** Theoretically, this algorithm is far superior in speed. It acts smartly by filtering which routes are worth exploring and discarding the ones that are obviously too far.
- **Bellman-Ford's Theory:** Theoretically, this algorithm is much slower because it forces itself to blindly re-evaluate and recalculate every single road over and over again, regardless of distance.

C. Empirical Study (A4)

To push the limits, the testing was conducted directly inside the web browser's engine by generating thousands of random location points. The execution time is measured in milliseconds (ms).

Vertices (Locations)	Edges (Roads)	Dijkstra Time (ms)	Bellman- Ford Time (ms)
100	1000	2.00	0.10
500	5000	12.20	0.50
1000	10000	41.60	1.00

2500	25000	289.50	3.50
5000	50000	1055.60	4.30

4. Conclusion

A. Summary of Findings (C1)

This project proves that classic graph algorithms are highly useful for solving logistics routing problems, such as navigating the map of Surabaya. The most valuable lesson learned is that theoretical algorithm speed on paper does not always match real-world software performance. The execution speed of a program is heavily influenced by the compatibility between the algorithm and the specific programming language environment being used.

Limitations & Future Work

The main limitation of this web application is the browser's user interface rendering load, where the browser begins to struggle if asked to process more than 5,000 location points simultaneously. In the future, this heavy calculation process should be moved to a dedicated backend server.